

设计模式	Design Patterns
ReAct	Reasoning + Acting 交替循环
Reflection	自我评审 + 重写,迭代提质
Reflexion	带长期记忆的反馈闭环
Plan-and-Execute	先规划任务树再逐步执行
Tree-of-Thoughts	多分支搜索 + 评分剪枝
Self-Ask	分解为子问题逐个解答
CoT / ToT / GoT	链 / 树 / 图 状思考
Routing	按意图分发到不同子链
Multi-Agent Supervisor	主管派发,worker 各司其职
Swarm	对等 handoff,共享 state
Hierarchical	多层分组,经理 + 员工
Debate	多 agent 辩论求共识
何时升级	单步可解 → 直接 prompt;多步 → ReAct;难且贵 → Plan;并发 → Multi-agent

Context Engineering	Context Eng.
窗口管理	
token budget	为每段预留配额
compaction	超阈值自动摘要重写
truncation	硬截断 / 头尾保留
隔离与分发	
sub-agents	独立 context,只回主结论
state passing	显式传字段,而非整个 history
scratchpad	中间思考,用完即丢
外部知识	
RAG	检索 + 重排 + 注入
tool result	按需拉取,避免预灌
citation	带源 + 可追溯
缓存与省钱	
prompt caching	系统/工具前缀复用,降 90% 成本
cache_control	Anthropic 显式断点标记
原则	"给最少够用的上下文" — 噪声越多,模型越容易跑偏 / 幻觉

核心组件	Core Components
LLM	推理引擎,决定下一步动作
Tools	外部函数 / API / MCP server
Memory	short-term + long-term
Planner	把目标拆成步骤
Executor	实际调用工具的循环
Critic / Reviewer	评估输出 / 触发重试
Router	条件分支 / 模型选择
State	跨节点共享的数据结构
Guardrails	输入输出校验 / 越权拦截
HITL	人在回路审批关键动作
控制流 vs 数据流	
控制流:	谁来决定下一步 (LLM / 代码 / 路由)。数据流: state 如何在节点间传递
Agent Loop 骨架	<pre>while not done: msg = llm(state) if msg.tool_calls: state += run_tools(msg) else: done = True</pre>

主流框架	Frameworks
LangGraph	图状 workflow,state machine
OpenAI Agents SDK	handoff + guardrails 轻量
Claude Agent SDK	Skills + MCP + sub-agents
AutoGen	微软 · 多 agent 对话
CrewAI	role-based 团队协作
LlamaIndex Agents	数据为中心 + RAG 强项
Pydantic AI	类型安全 / Python 风格
Mastra	TypeScript 全栈 agent
编排范式	
Graph	LangGraph · 显式节点边
Handoff	OpenAI / Swarm
Role	CrewAI · 拟人化分工
Code-first	Claude SDK · 写循环
选型	复杂分支 → LangGraph;简单线性 → OpenAI SDK;Claude 长任务 → Claude SDK

Tool Use	Function Calling
JSON Schema	工具签名 / 参数类型
parallel_tool_calls	同轮并发多工具
tool_choice	auto / required / 指定
strict: true	OpenAI 严格 schema 校验
error → tool_result	失败回填,LLM 自修
三家 schema 差异	
OpenAI	tools[].function.parameters
Anthropic	tools[].input_schema
Gemini	function_declarations
LangChain @tool	<pre>@tool def search(q: str) -> str: """Search the web for q.""" return ddg(q)</pre>
设计要点	名字动词化、描述写清楚何时该用、参数加 enum/range、返回结构化、副作用要 idempotent

评估 & 可观测	Eval & Observability
Trace 概念	
trace	一次完整 agent 运行
span	单次 LLM / tool 调用
OpenTelemetry	统一可观测协议
平台	
LangSmith	LangChain 官方,trace + eval
Langfuse	开源 self-host 友好
Arize Phoenix	开源 OTel 原生
Braintrust	eval + playground 强
Helicone / W&B	代理日志 / 实验追踪
评估指标	
success rate	任务完成率
tool call accuracy	工具选用 / 参数对错
latency p50 / p95	响应延迟分布
cost per task	token × 单价
LLM-as-judge	用模型打分,需校准
数据集	
线上 trace 沉淀为 eval set	回归 / A/B 模型升级

Memory 类型	Memory
Working	当前 context window 里的内容
Episodic	会话历史 / 过往交互
Semantic	事实知识 → RAG / vector DB
Procedural	技能 / skills / playbooks
Entity	关于人 / 物 的结构化记忆
压缩策略	
Summarization	老消息合并为摘要
Sliding window	只保留最近 N 轮
Compaction	触发阈值 → 重写 state
Hierarchical	分层摘要 (近期+长期)
存储后端	
Postgres + pgvector	混合存储常用方案
Redis	短期 / 会话 KV
Pinecone / Qdrant	专用向量库
写入时机	
每轮 / 会话结束 / 显式 save_memory	工具调用 / 后台 reflection

高效技巧	Pro Tips
强类型 schema:Pydantic / Zod 兜底,LLM 输出能直接进业务	
Tool 描述写清"何时该用 / 何时别用",比堆参数说明更有效	
主 agent 调度,子 agent 隔离上下文,只回结论不回 trace	
每轮检查循环计数,超过 N 步强制 break,防止死循环烧钱	
关键节点加 HITL(写库 / 转账 / 发邮件),先 confirm 再执行	
失败 tool_result 带 error 回填,让 LLM 自修而不是直接抛	
token 预算:每段(system / tools / history)定上限,溢出就 compact	
Anthropic cache_control 标系统 + 工具前缀,成本砍 90%	
避免 prompt 污染:用户输入用分隔符 / XML 包裹,别裸拼	
优先 parallel tool calls,串行只在依赖前一步结果时	
用 structured output + JSON mode 替代正则解析	
评估别只看 success rate,看 tool call accuracy + 成本	
线上 trace 直接喂回 eval set,回归升级模型才稳	
Multi-agent 慎用,单 agent + 工具能搞定就别上 — 沟通成本高	
给 agent 留 反思步骤:"再检查一遍" 常常救命	